实验二 作业调度实验

一. 目的要求：

用高级语言编写和调试一个或多个作业调度的模拟程序，以加深对作业调度算法的理解。

二. 例题：为单道批处理系统设计一个作业调度程序。

由于在单道批处理系统中，作业一投入运行，它就占有计算机的一切资源直到作业完成为止，因此调度作业时不必考虑它所需要的资源是否得到满足，它所占用的 CPU 时限等因素。

作业调度算法：采用先来先服务（FCFS）调度算法，即按作业提交的先后次序进行调度。总是首先调度在系统中等待时间最长的作业。

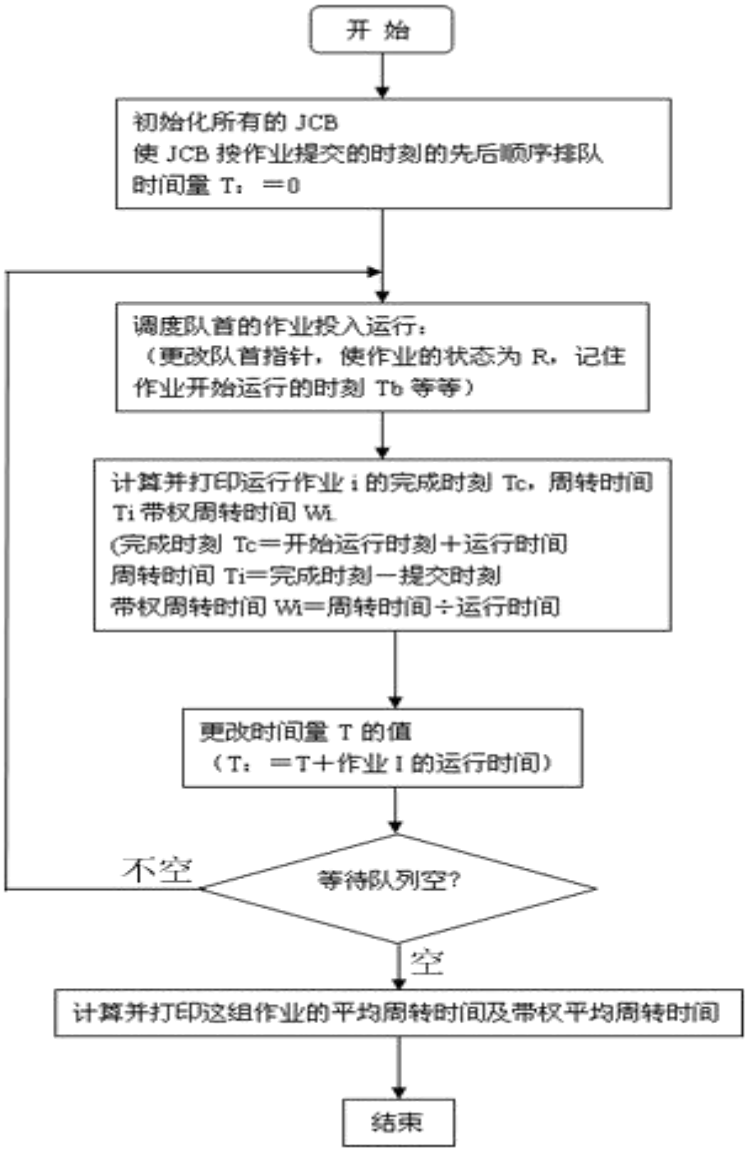
每个作业由一个作业控制块 JCB 表示，JCB 可以包含如下信息：**作业名、提交时间、所需的运行时间、所需的资源、作业状态、链指针等等。**

作业的状态可以是等待 W(Wait)、运行 R(Run)和完成 F(Finish)三种状态之一。每个作业的最初状态总是等待 W。

各个等待的作业按照提交时刻的先后次序排队，总是首先调度等待队列中队首的作业。

每个作业完成后要打印该作业的开始运行时刻、完成时刻、周转时间和带权周转时间，这一组作业完成后要计算并打印这组作业的平均周转时间、带权平均周转时间。

调度算法的流程图如下图所示。



三 ． 实习题：

1、编写并调试一个单道处理系统的作业等待模拟程序。

作业等待算法：分别采用先来先服务（FCFS），最短作业优先（SJF）、响应比高者优先（HRN）的调度算法。

对每种调度算法都要求打印每个作业开始运行时刻、完成时刻、周转时间、带权周转时间，以及这组作业的平均周转时间及带权平均周转时间，以比较各种算法的优缺点。

FCFS

```
#include <iostream>
using namespace std;

#define NEW() new JCB()

struct jcb{
    string name;
    int handInTime;
    int needTime;
    int startTime;
    int finishTime;
    int cycleTime;
    float weightCycleTime;
    string state;
    struct jcb *link;
} *p, *ready, *head;

typedef struct jcb JCB;

void input();
void run();
void print(JCB *);
void averageTime();

int amount,time;
```

```
// 输入作业信息
void input(){
    cout << "作业总数：";
    cin >> amount;
    head = NEW();
    JCB *q = head;
    for (int i = 0;i < amount;i++){
        p = NEW();
        cout << "\n作业No" << i << ":" << endl;
        cout << "作业名:";
        cin >> p->name;
        cout << "作业运行时间:";
        cin >> p->needTime;
        p->handInTime = time++;
        p->state = "W";
        p->link = NULL;
        q->link = p;
        q = q->link;
        p = p->link;
    }
}

// 模拟运行
void run(){
    ready->startTime = time;
    time += ready->needTime;
```

```
    ready->finishTime = time;
    ready->cycleTime = time - ready->handInTime;
    ready->weightCycleTime = ready->cycleTime / ready->needTime;
    ready->state = "F";
    print(ready);
}

// 作业完成后打印信息
void print(JCB* work){
    cout << endl;
    cout << "作业 [" << work->name << "] 已完成" << endl;
    cout << "完成时刻:" << work->finishTime << endl;
    cout << "周转时间:" << work->cycleTime << endl;
    cout << "带权周转时间:" << work->weightCycleTime << endl;
    cout << endl;
}

// 计算平均周转时间与带权平均周转时间并输出
void averageTime(){
    float averageCT, averageWCT;
    JCB *tmp = head->link;
    while(tmp != NULL){
        averageCT += tmp->cycleTime;
        averageWCT += tmp->weightCycleTime;
        tmp = tmp->link;
    }
```

```
    averageCT /= amount;
    averageWCT /= amount;

    printf("\n平均周转时间:%.2f\n",averageCT);
    printf("带权平均周转时间:%.2f\n\n", averageWCT);
}



// 主函数
int main(){
    head = ready = p = NULL;
    input();
    ready = head->link;
    while(ready != NULL){
        ready->state = "R";
        run();
        ready = ready->link;
        cout << "按任意键继续..." << endl;
        getchar();
    }
    averageTime();

    return 0;
}
```

```
作业No2:
作业名:c
作业运行时间:5

作业No3:
作业名:e
作业运行时间:2

作业No4:
作业名:f
作业运行时间:4

作业 [a] 已完成
完成时刻:9
周转时间:9
带权周转时间:2


作业 [b] 已完成
完成时刻:12
周转时间:11
带权周转时间:3


作业 [c] 已完成
完成时刻:17
周转时间:15
带权周转时间:3


作业 [e] 已完成
完成时刻:19
周转时间:16
带权周转时间:8


作业 [f] 已完成
完成时刻:23
周转时间:19
带权周转时间:4


平均周转时间:14.00
带权平均周转时间:4.00
```

SJF

```
   #include <iostream>
using namespace std;

#define NEW() new JCB()

struct jcb {
    string name;
    int handInTime;
    int needTime;
    int startTime;
    int finishTime;
    int cycleTime;
    float weightCycleTime;
    string state;
    struct jcb* link;
} *p, *ready, *head;

typedef struct jcb JCB;

void input();
void workSort();
void run();
void print(JCB*);
void averageTime();

int amount, time;

// 输入作业信息
void input() {
    cout << "作业总数：";
    cin >> amount;
    head = NEW();
    JCB* q = head;
    for (int i = 0; i < amount; i++) {
        p = NEW();
        cout << "\n作业No" << i << ":" << endl;
        cout << "作业名:";
        cin >> p->name;
        cout << "作业运行时间:";
        cin >> p->needTime;
        p->handInTime = time++;
        p->state = "W";
        p->link = NULL;
        q->link = p;
```

```
        q = q->link;
        p = p->link;
    }
}

// 根据作业长度排序
void workSort() {
    for (int i = 0; i < amount - 1; i++) {
        int num = amount - i - 1;
        JCB *p = head->link, *q = p->link, *tail = head;
        while (num--) {
            if (q->needTime < p->needTime) {
                p->link = q->link;
                q->link = p;
                tail->link = q;
                q = q->link->link;
            } else {
                q = q->link;
                p = p->link;
            }
            tail = tail->link;
        }
    }
}

// 模拟运行
void run() {
    ready->startTime = time;
    time += ready->needTime;
    ready->finishTime = time;
    ready->cycleTime = time - ready->handInTime;
    ready->weightCycleTime = ready->cycleTime / ready->needTime;
    ready->state = "F";
    print(ready);
}

// 作业完成后打印信息
void print(JCB* work) {
    cout << endl;
    cout << "作业 [" << work->name << "] 已完成" << endl;
    cout << "完成时刻:" << work->finishTime << endl;
    cout << "周转时间:" << work->cycleTime << endl;
    cout << "带权周转时间:" << work->weightCycleTime << endl;
    cout << endl;
```

```
}

// 计算平均周转时间与带权平均周转时间并输出
void averageTime() {
    float averageCT, averageWCT;
    JCB* tmp = head->link;
    while (tmp != NULL) {
        averageCT += tmp->cycleTime;
        averageWCT += tmp->weightCycleTime;
        tmp = tmp->link;
    }
    averageCT /= amount;
    averageWCT /= amount;

    printf("\n平均周转时间:%.2f\n", averageCT);
    printf("带权平均周转时间:%.2f\n\n", averageWCT);
}

// 主函数
int main() {
    head = ready = p = NULL;
    input();
    workSort();
    ready = head->link;
    while (ready != NULL) {
        ready->state = "R";
        run();
        ready = ready->link;
        // cout << "按任意键继续..." << endl;
        // getchar();
    }
    averageTime();

    return 0;
}
```

```
作业No2:
作业名:c
作业运行时间:5

作业No3:
作业名:e
作业运行时间:2

作业No4:
作业名:f
作业运行时间:4

作业 [e] 已完成
完成时刻:7
周转时间:4
带权周转时间:2


作业 [b] 已完成
完成时刻:10
周转时间:9
带权周转时间:3


作业 [a] 已完成
完成时刻:14
周转时间:14
带权周转时间:3


作业 [f] 已完成
完成时刻:18
周转时间:14
带权周转时间:3


作业 [c] 已完成
完成时刻:23
周转时间:21
带权周转时间:4


平均周转时间:12.40
带权平均周转时间:3.00
```

HRN

```
   #include <iostream>
using namespace std;

#define NEW() new JCB()

struct jcb {
    string name;
    int handInTime;
    int needTime;
    int startTime;
    int finishTime;
    int cycleTime;
    int waitTime;
    float weightCycleTime;
    float super;
    string state;
    struct jcb* link;
} *p, *ready, *head;

typedef struct jcb JCB;

void input();
void workSort();
void run();
void print(JCB*);
void averageTime();

int amount, time;

// 输入作业信息
void input() {
    cout << "作业总数：";
    cin >> amount;
    head = NEW();
    JCB* q = head;
    for (int i = 0; i < amount; i++) {
        p = NEW();
        cout << "\n作业No" << i << ":" << endl;
        cout << "作业名:";
        cin >> p->name;
        cout << "作业运行时间:";
        cin >> p->needTime;
        p->handInTime = time++;
        p->waitTime = amount-i-1;
```

```
        p->state = "W";
        p->link = NULL;
        q->link = p;
        q = q->link;
        p = p->link;
    }
}

// 根据作业长度排序
void workSort() {
    float minn = 0x3f3f3f3f;
    p = head->link;
    while(p!=NULL){
        p->super = (float)(p->waitTime + p->needTime) / p->needTime;
        if(minn > p->super && p->state == "W"){
            minn = p->super;
            ready = p;
        }
        p = p->link;
    }
}

// 模拟运行
void run() {
    ready->startTime = time;
    time += ready->needTime;
    ready->finishTime = time;
    ready->cycleTime = time - ready->handInTime;
    ready->weightCycleTime = ready->cycleTime / ready->needTime;
    ready->state = "F";

    p = head->link;
    while(p!=NULL){
        if(p->state == "W") p->waitTime++;
        p = p->link;
    }
    print(ready);
}

// 作业完成后打印信息
void print(JCB* work) {
    cout << endl;
    cout << "作业 [" << work->name << "] 已完成" << endl;
    cout << "完成时刻:" << work->finishTime << endl;
```

```
    cout << "周转时间:" << work->cycleTime << endl;
    cout << "带权周转时间:" << work->weightCycleTime << endl;
    cout << endl;
}

// 计算平均周转时间与带权平均周转时间并输出
void averageTime() {
    float averageCT = 0, averageWCT = 0;
    JCB* tmp = head->link;
    while (tmp != NULL) {
        averageCT += tmp->cycleTime;
        averageWCT += tmp->weightCycleTime;
        tmp = tmp->link;
    }
    averageCT /= amount;
    averageWCT /= amount;

    printf("\n平均周转时间:%.2f\n", averageCT);
    printf("带权平均周转时间:%.2f\n\n", averageWCT);
}


// 主函数
int main() {
    head = ready = p = NULL;
    input();
    workSort();
    ready = head->link;
    for (int i = 0;i < amount;i++) {
        ready->state = "R";
        run();
        workSort();
        // cout << "按任意键继续..." << endl;
        // getchar();
    }
    averageTime();

    return 0;
}
```

```
作业No2:
作业名:c
作业运行时间:5

作业No3:
作业名:d
作业运行时间:2

作业No4:
作业名:f
作业运行时间:4

作业 [a] 已完成
完成时刻:9
周转时间:9
带权周转时间:2


作业 [f] 已完成
完成时刻:13
周转时间:9
带权周转时间:2


作业 [c] 已完成
完成时刻:18
周转时间:16
带权周转时间:3


作业 [b] 已完成
完成时刻:21
周转时间:20
带权周转时间:6


作业 [d] 已完成
完成时刻:23
周转时间:20
带权周转时间:10


平均周转时间:14.80
带权平均周转时间:4.60
```

2、编写并调度一个多道程序系统的作业调度模拟程序。

作业调度算法：采用基于先来先服务的调度算法。可以参考课本中的方法进行设计。

对于多道程序系统，要假定系统中具有的各种资源及数量、调度作业时必须考虑到每个作业的资源要求。

```
   #include<iostream>

#include<vector>

using namespace std;



typedef struct{

    string name;

    int handInTime;

    int needTime;

    int runTime;

    int needA,needB,needC;

    int startTime;

    int finishTime;

    int cycleTime;

    float weightCycleTime;

    string state;

}JCB;



vector<JCB> w,r,f;



void input();
```

```
void run();
void print(JCB);
void averageTime();

int sourceA = 7, sourceB = 8, sourceC = 6;
int amount, time;

// 输入作业信息
void input(){
    cout << "作业总数:";
    cin >> amount;
    JCB temp;
    for (int i = 0; i < amount; i++) {
        cout << "\n 作业 No" << i << ":" << endl;
        cout << "作业名:";
        cin >> temp.name;
        cout << "作业运行时间:";
        cin >> temp.needTime;
        cout << "作业所需资源量:";
        cin >> temp.needA >> temp.needB >> temp.needC;
        temp.handInTime = time++;
        temp.runTime = 0;
        temp.state = "W";
        w.push_back(temp);
```

```
    }
}

// 模拟运行
void run(){
    while(w.size() != 0){
         time++;
        for (int j = 0;j < r.size();j++){
            r[j].runTime++;
            if(r[j].runTime == r[j].needTime){
                r[j].state = "F";
                r[j].finishTime = time;
                r[j].cycleTime = time - r[j].startTime;
                r[j].weightCycleTime = r[j].cycleTime / r[j].needTime;
                sourceA += r[j].needA;
                sourceB += r[j].needB;
                sourceC += r[j].needC;
                print(r[j]);
                f.push_back(r[j]);

                r.erase(r.begin()+j);
                j--;
            }
        }
```

```
        if (sourceA >= w[0].needA && sourceB >= w[0].needB && sourceC >=
w[0].needC){

            w[0].state = "R";

            w[0].startTime = time;

            r.push_back(w[0]);

            sourceA -= w[0].needA;

            sourceB -= w[0].needB;

            sourceC -= w[0].needC;

            w.erase(w.begin());

        }

    }

    while(r.size() != 0){

        time++;

        for (int j = 0; j < r.size(); j++) {

            r[j].runTime++;

            if (r[j].runTime == r[j].needTime) {

                r[j].state = "F";

                r[j].finishTime = time;

                r[j].cycleTime = time - r[j].startTime;

                r[j].weightCycleTime = r[j].cycleTime / r[j].needTime;

                sourceA += r[j].needA;

                sourceB += r[j].needB;

                sourceC += r[j].needC;

                print(r[j]);
```

```
                f.push_back(r[j]);

                r.erase(r.begin() + j);
                j--;
            }
        }
    }
}

// 作业完成后打印信息
void print(JCB work) {
    cout << endl;
    cout << "作业 [" << work.name << "] 已完成" << endl;
    cout << "完成时刻:" << work.finishTime << endl;
    cout << "周转时间:" << work.cycleTime << endl;
    cout << "带权周转时间:" << work.weightCycleTime << endl;
    cout << endl;
}

// 计算平均周转时间与带权平均周转时间并输出
void averageTime() {
    float averageCT, averageWCT;

    for (int i = 0;i < f.size();i++){
```

```
        averageCT += f[i].cycleTime;
        averageWCT += f[i].weightCycleTime;
    }
    averageCT /= amount;
    averageWCT /= amount;

    printf("\n平均周转时间:%.2f\n", averageCT);
    printf("带权平均周转时间:%.2f\n\n", averageWCT);
}

int main(){
    input();
    run();
    averageTime();

    return 0;
}
```

```
作业总数:3

作业No0:
作业名:a
作业运行时间:3
作业所需资源量:3 4 5

作业No1:
作业名:b
作业运行时间:2
作业所需资源量:5 5 5

作业No2:
作业名:c
作业运行时间:2
作业所需资源量:1 1 1

作业 [a] 已完成
完成时刻:7
周转时间:3
带权周转时间:1


作业 [b] 已完成
完成时刻:9
周转时间:2
带权周转时间:1


作业 [c] 已完成
完成时刻:10
周转时间:2
带权周转时间:1


平均周转时间:2.33
带权平均周转时间:1.00
```

3、编写并调试一个多道程序系统的作业调度模拟程序。

作业调度算法：采用基于优先级的作业调度。

可以参考课本中的例子自行设计。

```
#include <iostream>
```

```
#include <vector>
#include <algorithm>
using namespace std;

typedef struct {
    string name;
    int handInTime;
    int needTime;
    int runTime;
    int needA, needB, needC;
    int startTime;
    int finishTime;
    int cycleTime;
    int super;
    float weightCycleTime;
    string state;
} JCB;

vector<JCB> w, r, f;

void input();
void run();
void print(JCB);
void averageTime();
```

```
int sourceA = 7, sourceB = 8, sourceC = 6;
int amount, time;

bool cmp(const JCB& a, const JCB& b) {
    return a.super > b.super;
}

// 输入作业信息
void input() {
    cout << "作业总数:";
    cin >> amount;
    JCB temp;
    for (int i = 0; i < amount; i++) {
        cout << "\n作业No" << i << ":" << endl;
        cout << "作业名:";
        cin >> temp.name;
        cout << "作业优先度:";
        cin >> temp.super;
        cout << "作业运行时间:";
        cin >> temp.needTime;
        cout << "作业所需资源量:";
        cin >> temp.needA >> temp.needB >> temp.needC;
        temp.handInTime = time++;
```

```
        temp.runTime = 0;
        temp.state = "W";
        w.push_back(temp);
    }
    sort(w.begin(),w.end(),cmp);
}

// 模拟运行
void run() {
    while (w.size() != 0) {
        time++;
        for (int j = 0; j < r.size(); j++) {
            r[j].runTime++;
            if (r[j].runTime == r[j].needTime) {
                r[j].state = "F";
                r[j].finishTime = time;
                r[j].cycleTime = time - r[j].startTime;
                r[j].weightCycleTime = r[j].cycleTime / r[j].needTime;
                sourceA += r[j].needA;
                sourceB += r[j].needB;
                sourceC += r[j].needC;
                print(r[j]);
                f.push_back(r[j]);
```

```
                r.erase(r.begin() + j);
                j--;
            }
        }
        if (sourceA >= w[0].needA && sourceB >= w[0].needB &&
            sourceC >= w[0].needC) {
            w[0].state = "R";
            w[0].startTime = time;
            r.push_back(w[0]);
            sourceA -= w[0].needA;
            sourceB -= w[0].needB;
            sourceC -= w[0].needC;
            w.erase(w.begin());
        }
    }
    while (r.size() != 0) {
        time++;
        for (int j = 0; j < r.size(); j++) {
            r[j].runTime++;
            if (r[j].runTime == r[j].needTime) {
                r[j].state = "F";
                r[j].finishTime = time;
                r[j].cycleTime = time - r[j].startTime;
                r[j].weightCycleTime = r[j].cycleTime / r[j].needTime;
```

```
                sourceA += r[j].needA;
                sourceB += r[j].needB;
                sourceC += r[j].needC;
                print(r[j]);
                f.push_back(r[j]);

                r.erase(r.begin() + j);
                j--;
            }
        }
    }
}

// 作业完成后打印信息
void print(JCB work) {
    cout << endl;
    cout << "作业 [" << work.name << "] 已完成" << endl;
    cout << "完成时刻:" << work.finishTime << endl;
    cout << "周转时间:" << work.cycleTime << endl;
    cout << "带权周转时间:" << work.weightCycleTime << endl;
    cout << endl;
}

// 计算平均周转时间与带权平均周转时间并输出
```

```
void averageTime() {
    float averageCT, averageWCT;

    for (int i = 0; i < f.size(); i++) {
        averageCT += f[i].cycleTime;
        averageWCT += f[i].weightCycleTime;
    }
    averageCT /= amount;
    averageWCT /= amount;

    printf("\n平均周转时间:%.2f\n", averageCT);
    printf("带权平均周转时间:%.2f\n\n", averageWCT);
}

int main() {
    input();
    run();
    averageTime();

    return 0;
}
```

```
作业总数:3

作业No0:
作业名:a
作业优先度:3
作业运行时间:4
作业所需资源量:3 4 5

作业No1:
作业名:b
作业优先度:2
作业运行时间:3
作业所需资源量:1 1 1

作业No2:
作业名:c
作业优先度:4
作业运行时间:3
作业所需资源量:6 6 6

作业 [c] 已完成
完成时刻:7
周转时间:3
带权周转时间:1


作业 [a] 已完成
完成时刻:11
周转时间:4
带权周转时间:1


作业 [b] 已完成
完成时刻:11
周转时间:3
带权周转时间:1


平均周转时间:3.33
带权平均周转时间:1.00
```